

Señales Analógicas y ADC: NUCLEO-F446RE y Node-RED

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Resumen de la clase:

- 3.1 Temas para la semana 3.
- 3.2 Repaso: ADC.
- 3.3 Configuración ADC con CUBE MX.
- 3.4 Uso de las herramientas HAL.
- 3.5 Debug y Node-RED.
- 3.6 Problemas.

Temas para la semana 3

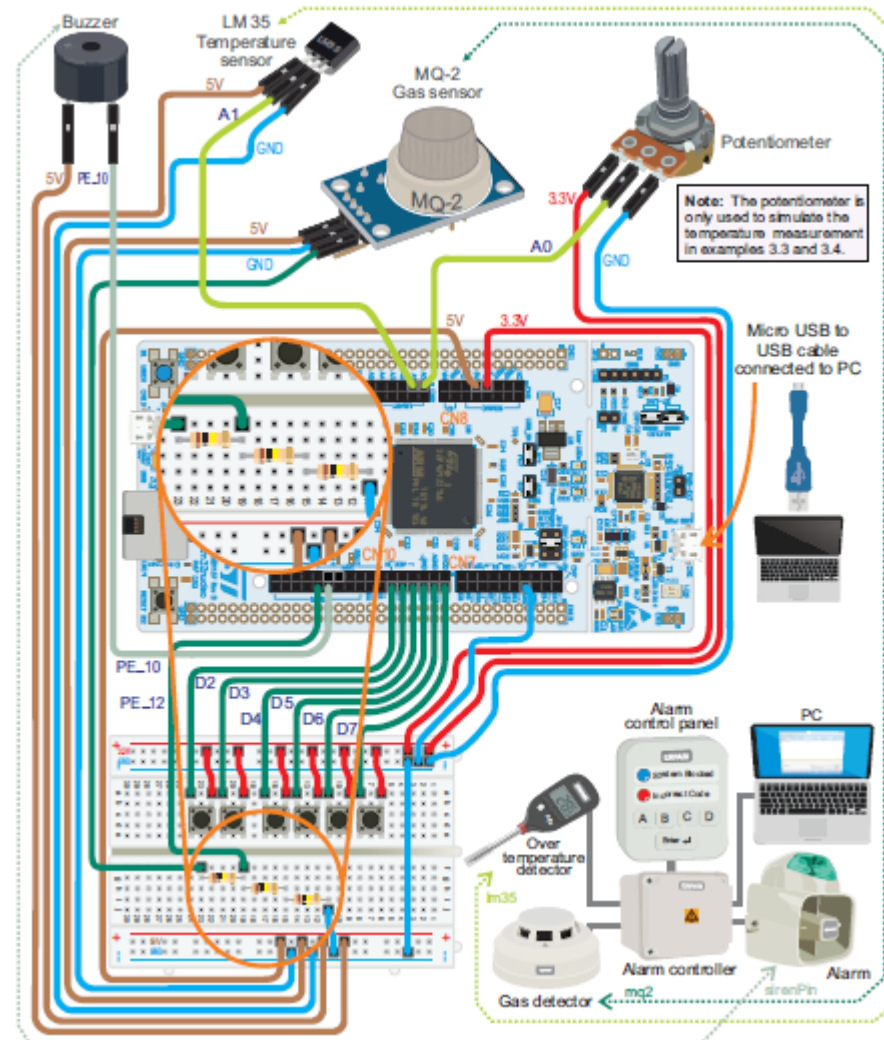


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.1 Temas para la semana 3.

3	Contenido de las clases de la semana 3. Teoría: Fundamentos de ADC. Uso de sensores analógicos y potenciómetros. Práctica: Lectura de señales analógicas. Creación de medidores y gráficos en tiempo real para representar los valores del ADC enviados por la placa NUCLEO-F446RE.	Será informada por el docente a cargo.	1, 2, 3 y 4.
4	Contenido de las clases de la semana 4. Teoría: Temporizadores (Timers) y sus registros de control. Interrupciones externas. Concepto de frecuencia. PWM. Práctica: Generación de señales PWM. Implementación de retardos y generación de tiempos periódicos. Uso de botones por interrupción. Interfaz Node-RED y placa NUCLEO-F446RE, haciendo uso de interrupciones externas.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Repaso: ADC

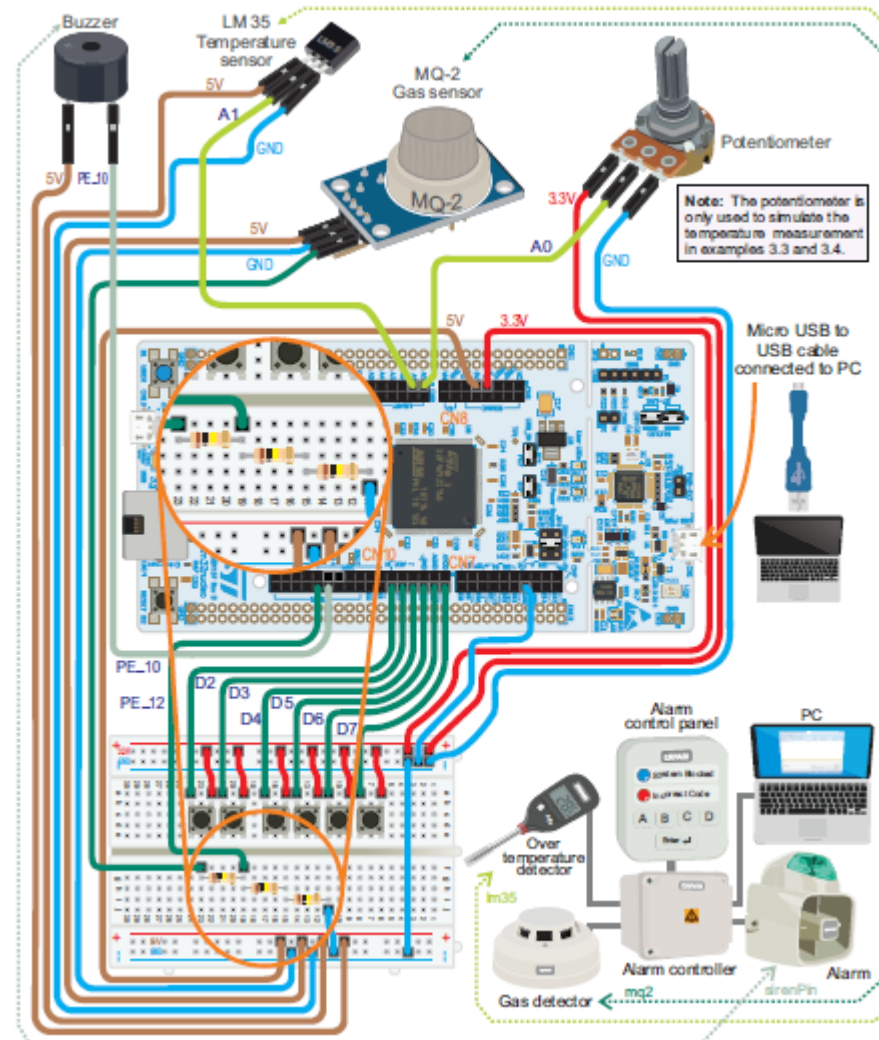
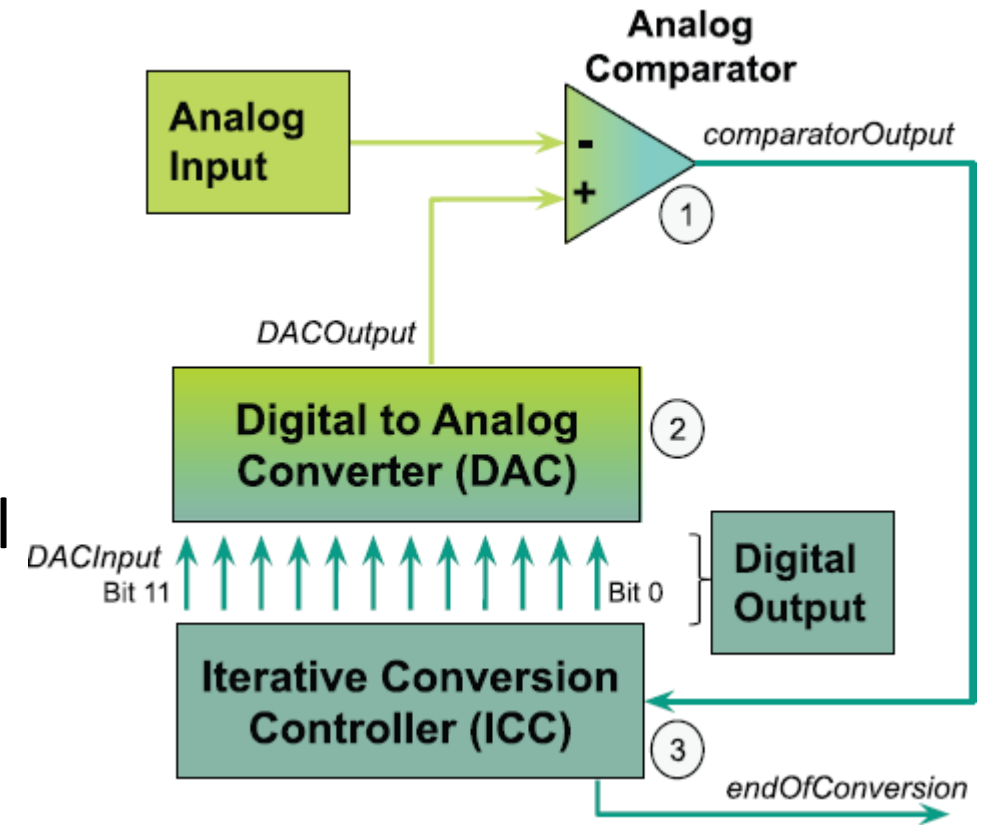


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

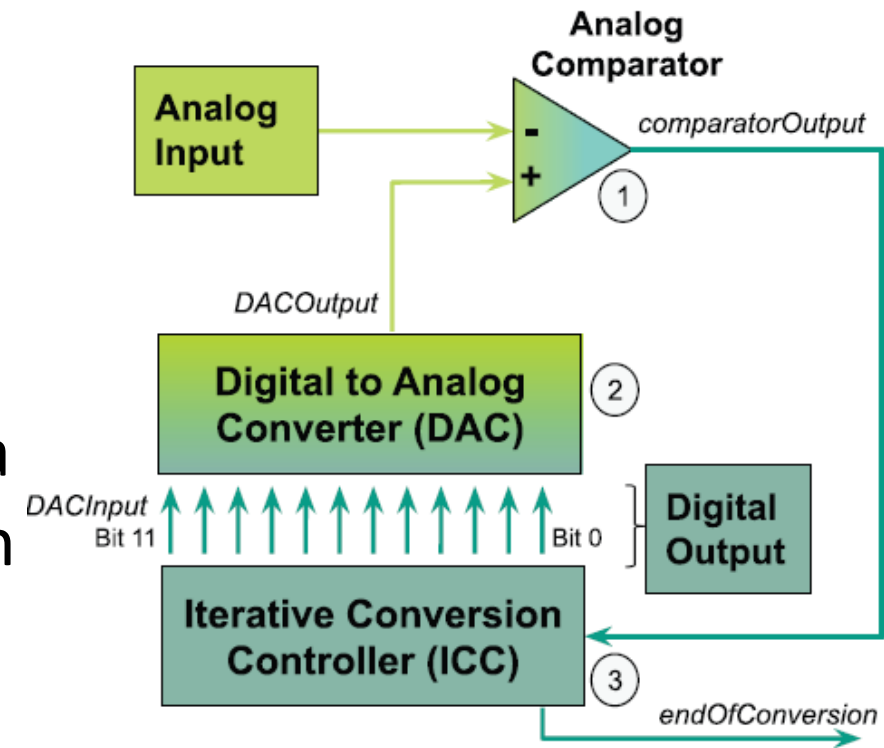
3.2 Repaso: ADC.

- El Conversor Analógico-Digital (ADC) realiza proceso que convierte una señal analógica en una señal digital.
- Un conversor ADC genera un número digital (binario) que representa una aproximación de la magnitud del voltaje de entrada.
- Es uno de los periféricos más complejos y con mayor número de configuraciones en un microcontrolador.



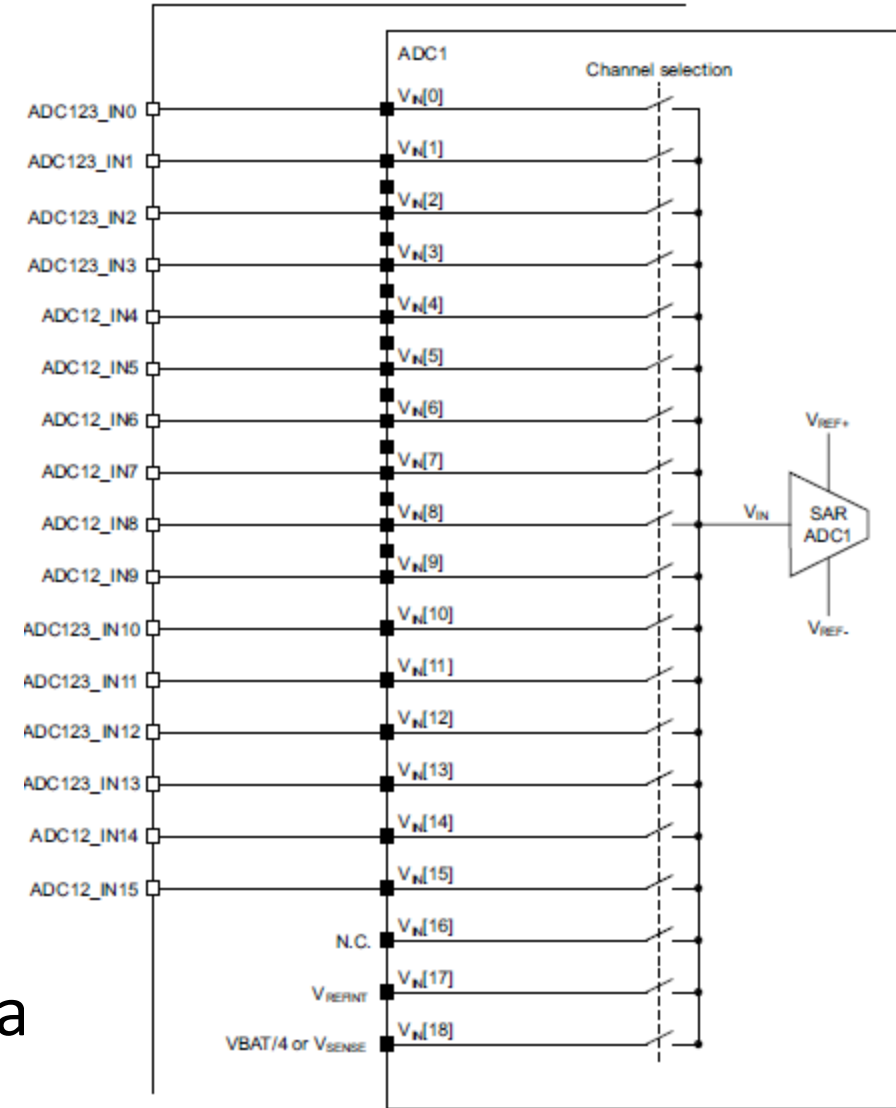
3.2 Repaso: ADC.

- El ADC integrado en los STM32 se basa en la arquitectura SAR (Successive Approximation Register) o de aproximaciones sucesivas.
- El proceso de conversión sigue pasos iterativos comparando el voltaje de entrada con un DAC interno hasta determinar el valor digital exacto bit a bit.

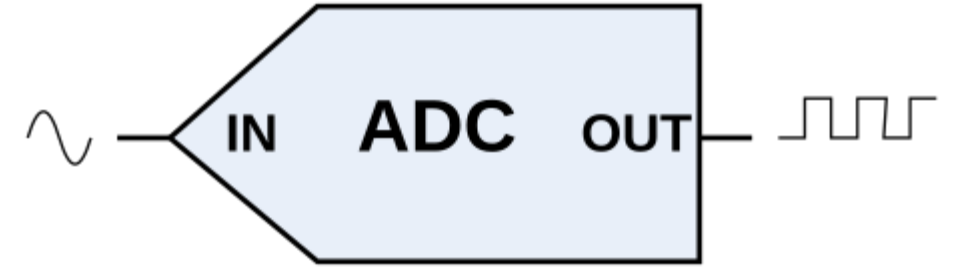


3.2 Repaso: ADC.

- El microcontrolador STM32F446RE incorpora **3 ADCs independientes**.
- **Resolución de 12 bits y 16 canales.**
- Manejo eficiente de datos mediante DMA (Acceso Directo a Memoria), reduce la carga CPU.
- El tiempo de muestreo es programable por canal, optimizando la relación precisión vs velocidad de conversión.

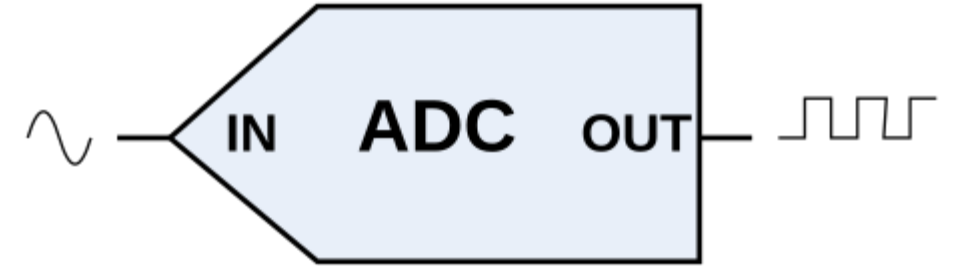


3.2 Repaso: ADC.



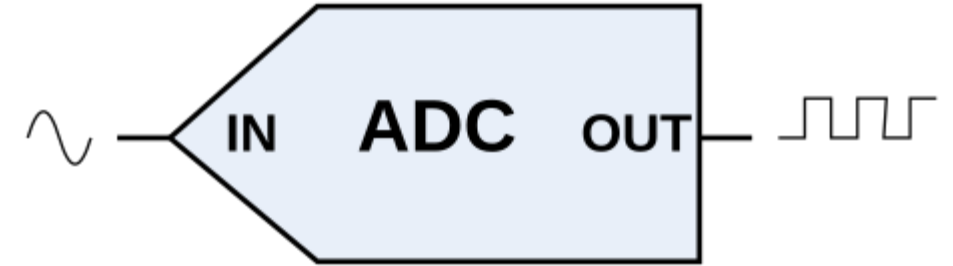
- Cuenta con hasta 19 canales multiplexados, lo que le permite medir señales de 16 fuentes externas, dos fuentes internas y el canal VBAT.
- La conversión A/D de los canales puede realizarse en *modo simple*, continuo, de barrido o discontinuo.
- Su resultado se almacena en un registro de datos de 16 bits alineado a la izquierda o a la derecha.
- Vigilancia analógica: permite detectar si la tensión de entrada supera los umbrales definidos por el usuario (superior o inferior).

3.2 Repaso: ADC.



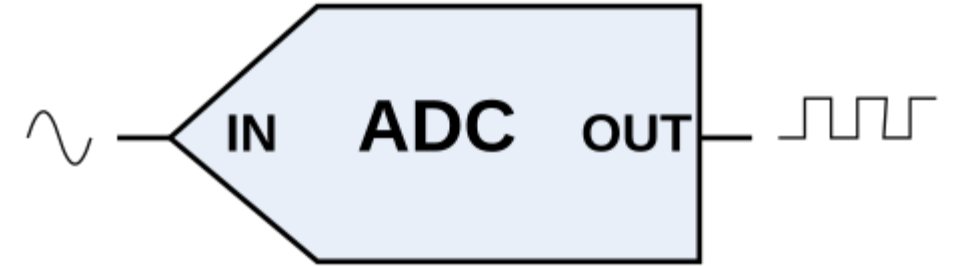
- Resolución configurable de 12, 10, 8 o 6 bits.
- Generación de interrupciones: al final de la conversión, en caso de sobrecorriente o del temporizador de vigilancia analógico.
- Modos de conversión simple y continua.
- Modo de escaneo para el ADC automático del canal 0 al canal 'n'.

3.2 Repaso: ADC.



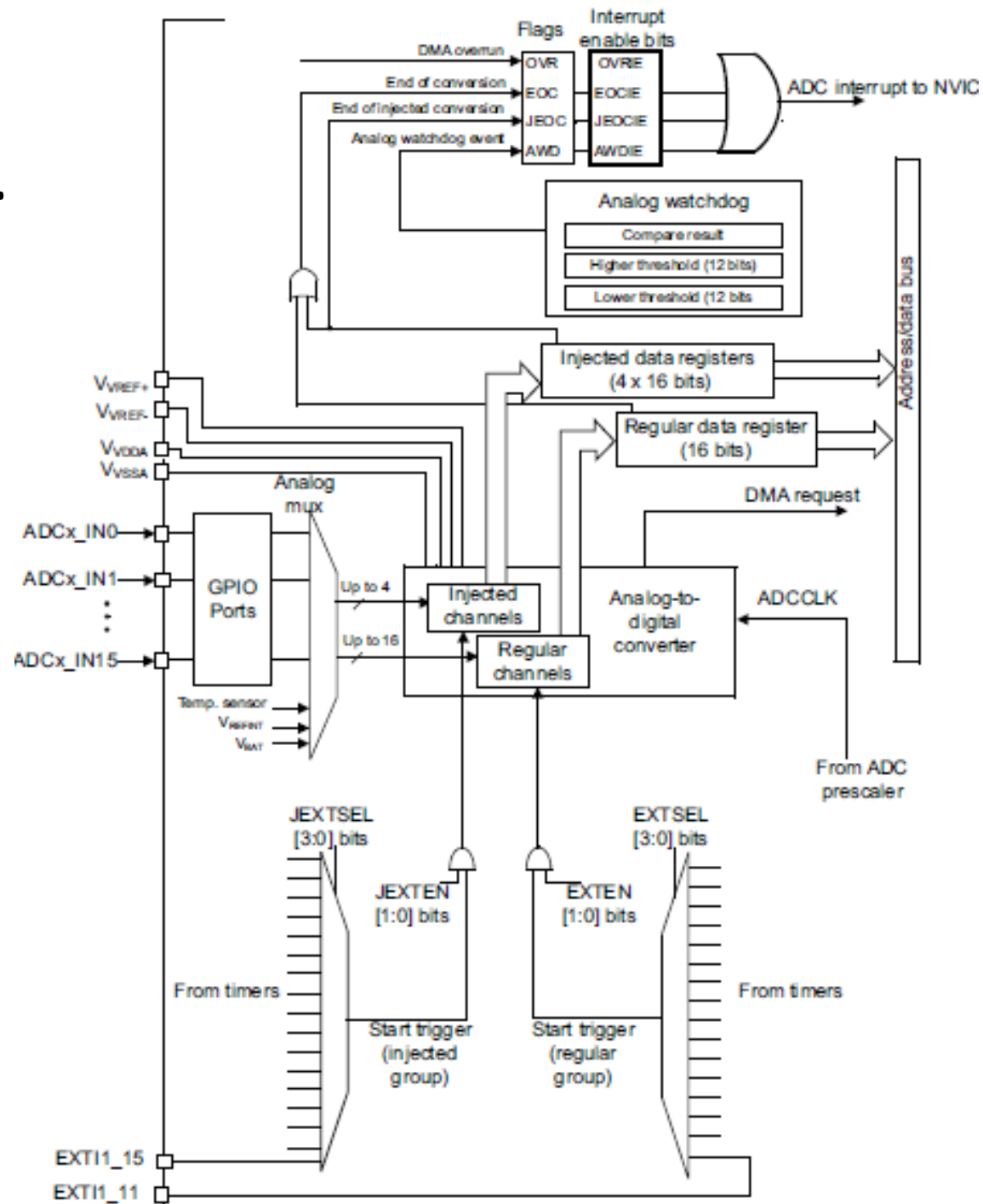
- Alineación de datos con coherencia de datos integrada.
- Tiempo de muestreo programable por canal.
- Opción de disparo externo con polaridad configurable para conversiones regulares e inyectadas.
- Modo discontinuo.
- ***Modo dual/triple (en dispositivos con 2 o más ADC).***

3.2 Repaso: ADC.



- Almacenamiento de datos DMA configurable en modo ADC dual/triple.
- Retardo configurable entre conversiones en modo entrelazado dual/triple.
- Rango de entrada del ADC: $V_{REF-} \leq V_{IN} \leq V_{REF+}$.
- Generación de solicitudes DMA durante la conversión de canal regular.

3.2 Repaso: ADC.



Configuración ADC con CUBE MX

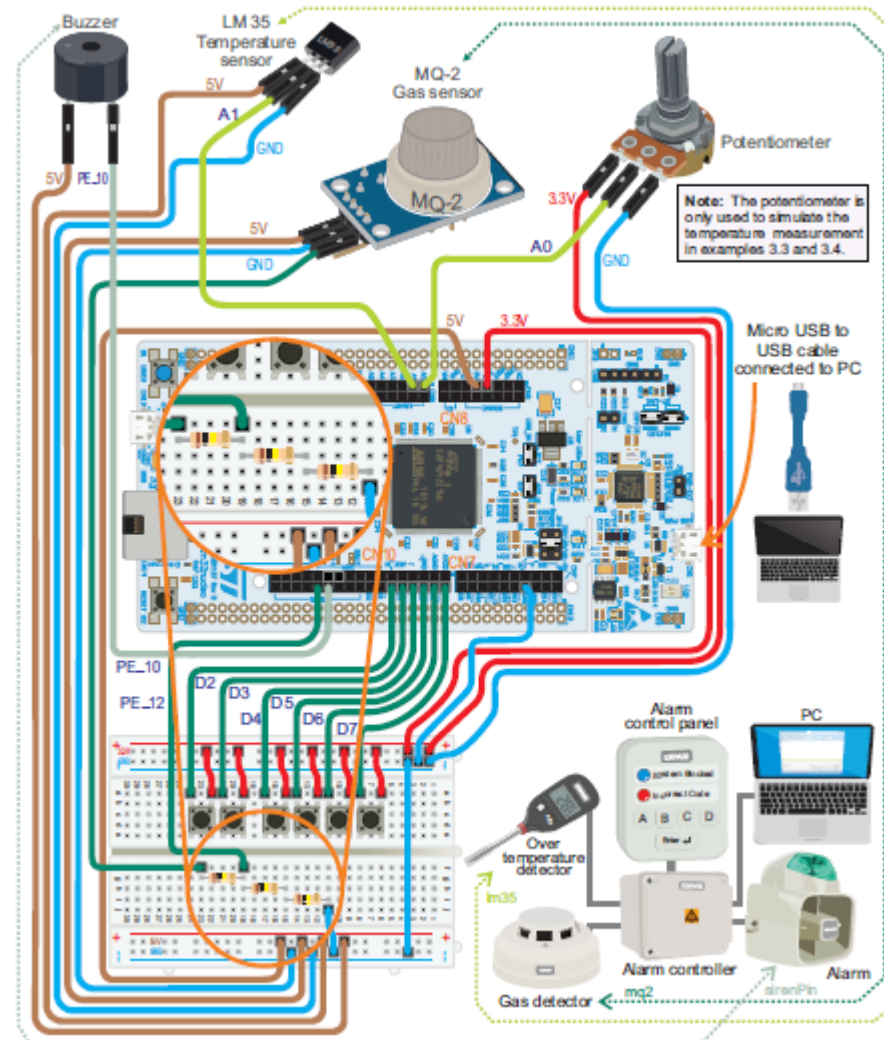
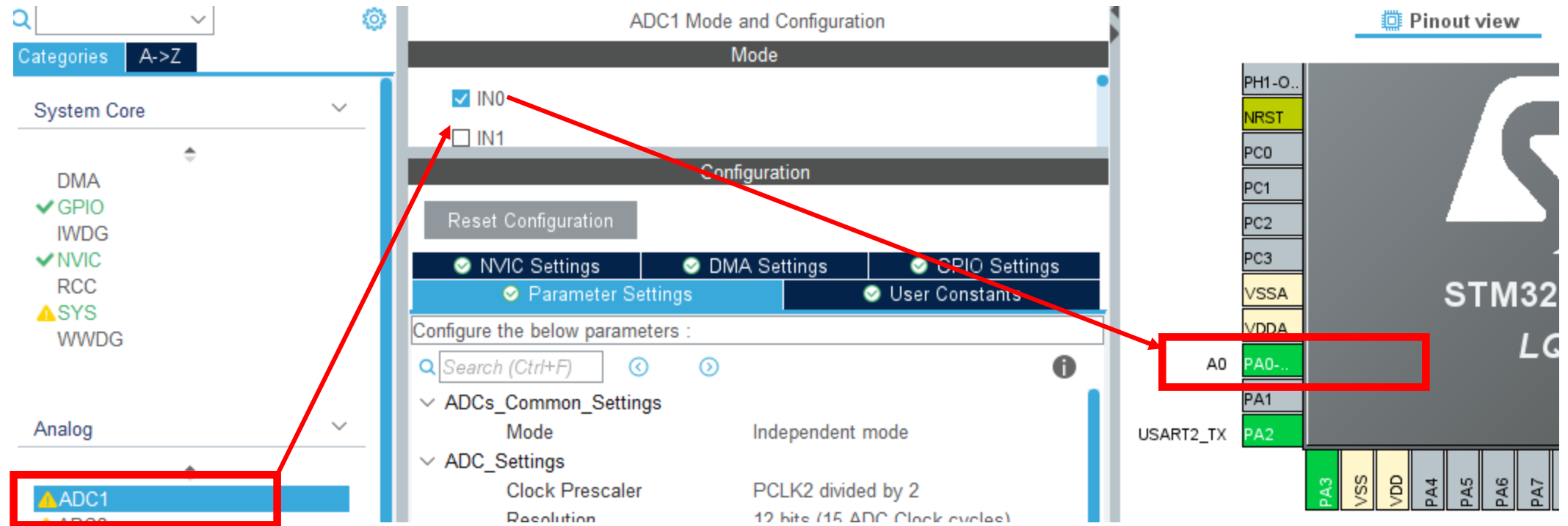


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

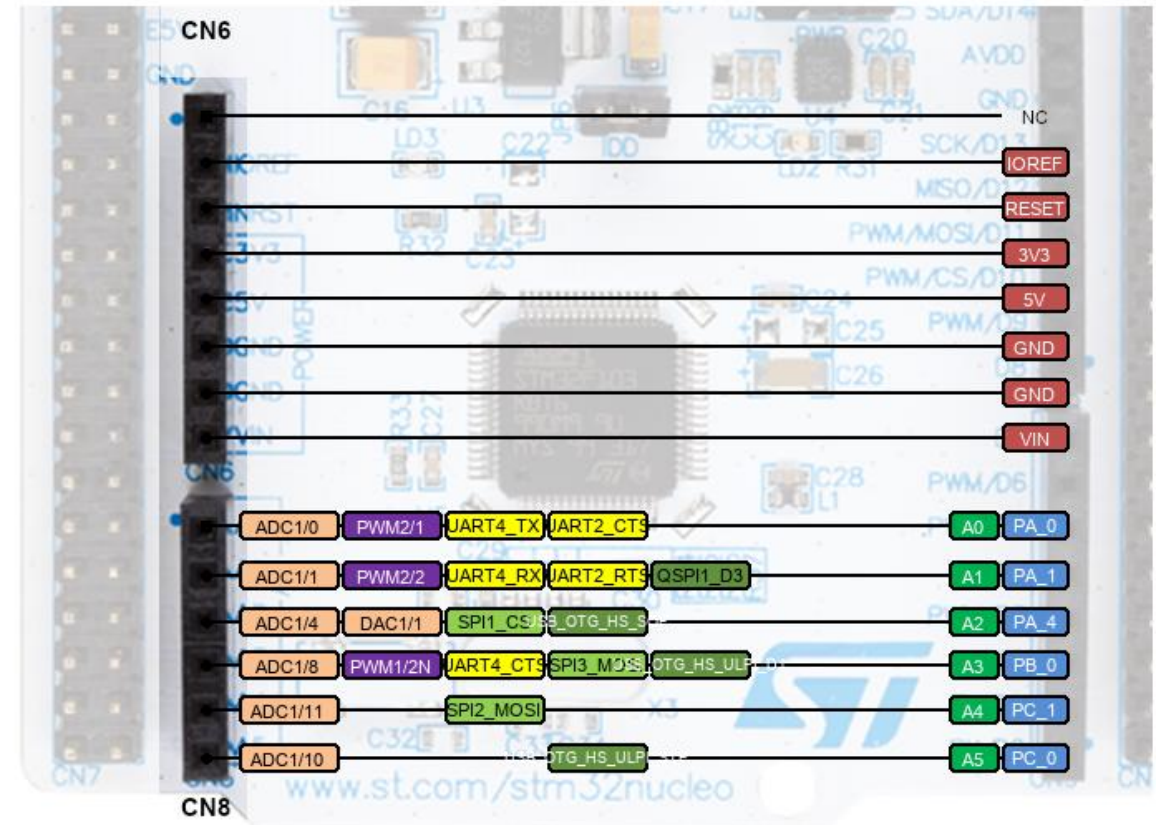
3.3 Configuración ADC con CUBE MX

- Configuración del pin de entrada al ADC1.



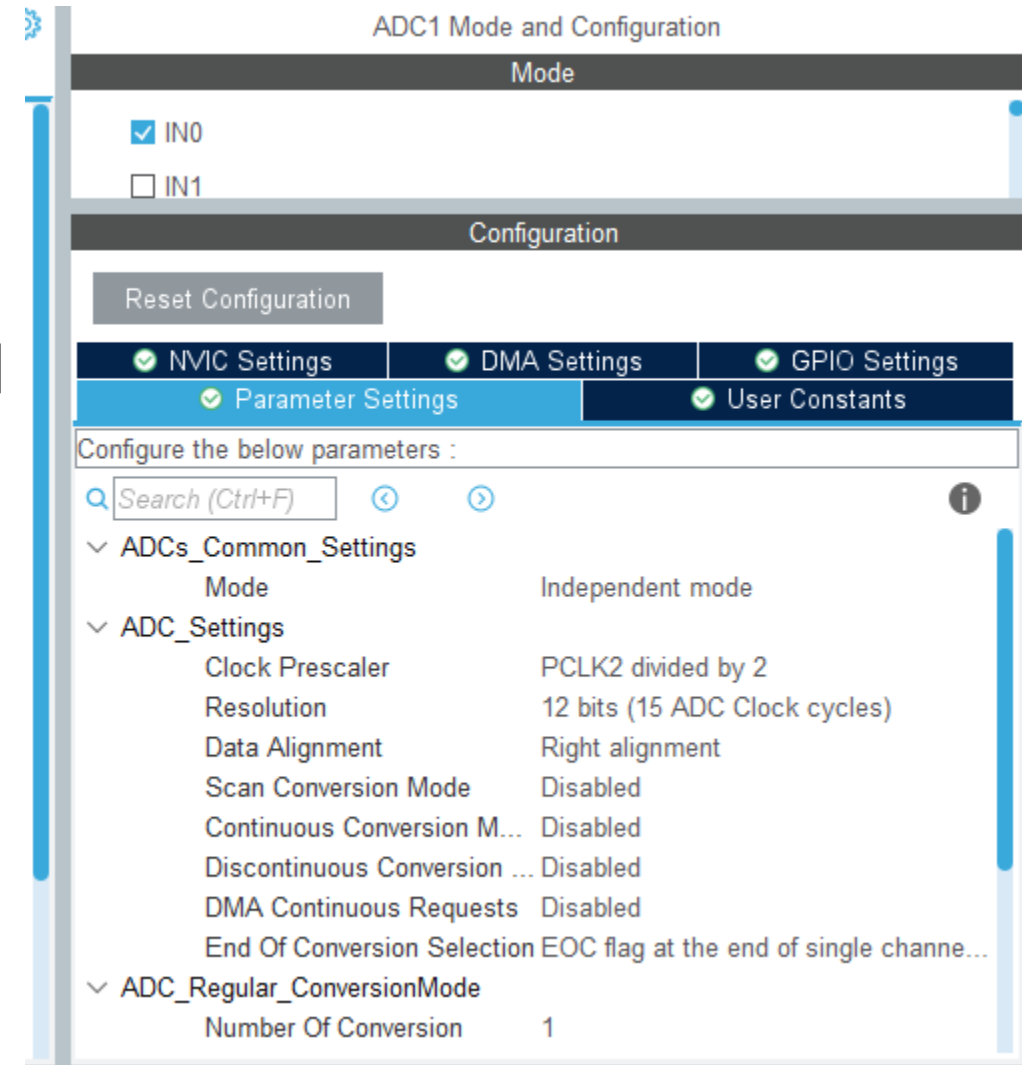
3.3 Configuración ADC con CUBE MX

- En la diapositiva se observa los pines que se pueden configurar como una entrada para el ADC en la placa núcleo.



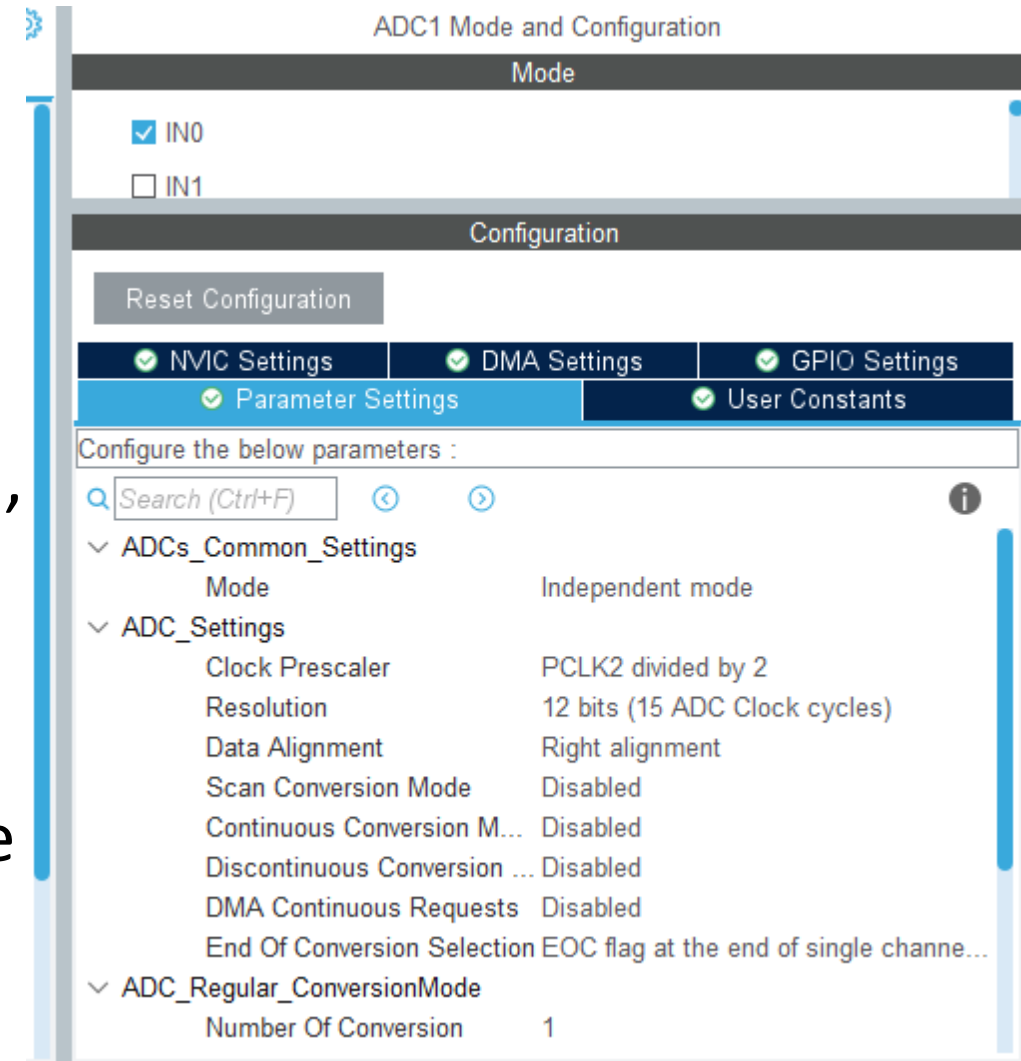
3.3 Configuración ADC con CUBE MX

- Significado de los parámetros:
- Clock Prescaler: Divide la frecuencia del reloj que llega al ADC.
- En "By 2" significa que el ADC funciona a la mitad de la frecuencia del bus APB.
- Resolution: 12 bits significa que el voltaje se divide en $2^{12} = 4096$ niveles (12 bits es la máxima precisión).



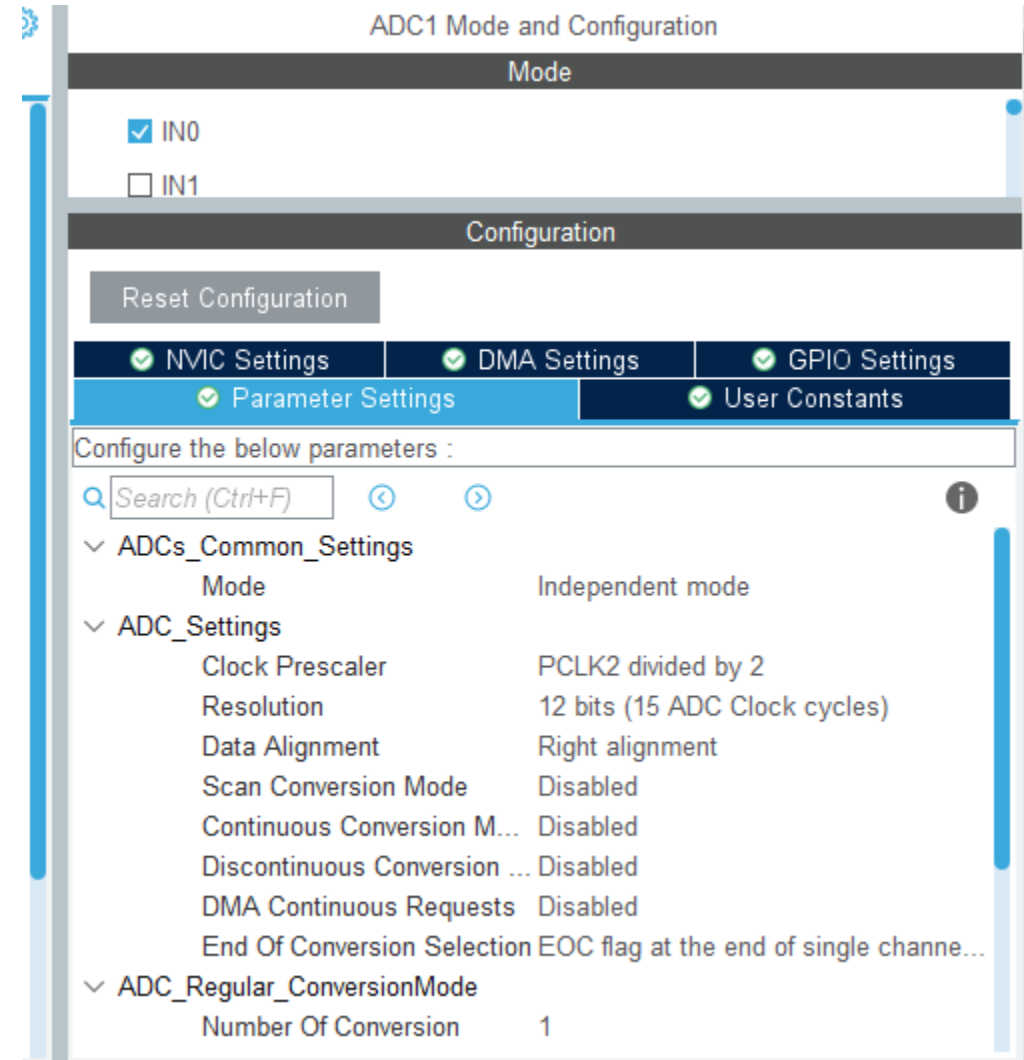
3.3 Configuración ADC con CUBE MX

- Data Alignment: El registro donde se guarda el dato es de 16 bits.
- Significado: "Right" pone el dato al final, es lo estándar.
- Scan Conversion Mode: Si se habilitan varios canales (IN0, IN1, etc.), esto hace que el ADC los recorra en fila.
- Disabled si solo usas un canal, Enabled si usas varios.



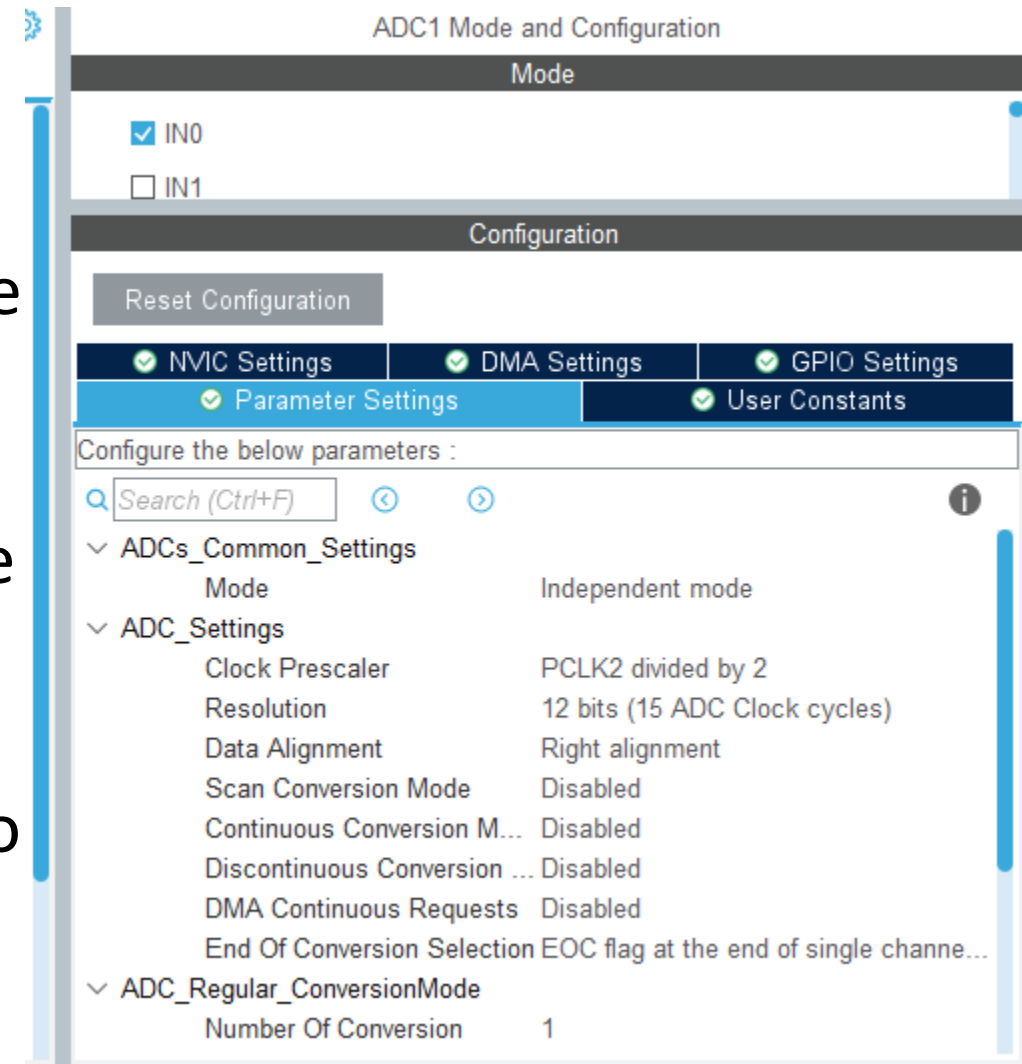
3.3 Configuración ADC con CUBE MX

- Continuous Conversion: para Polling usar en DISABLED.
- **Significado: El ADC hace una sola conversión y se detiene.**
- Si está en ENABLED: En cuanto termina una conversión, el hardware arranca la siguiente automáticamente.



3.3 Configuración ADC con CUBE MX

- Discontinuous Conversion: Permite que el ADC convierta solo un **grupo** corto de canales tras un trigger.
- DMA Continuous: Mantiene al asistente DMA moviendo datos sin parar.
- End of Conversion (EOC): Define cuándo se levanta la "bandera" de finalizado.
- **Para un solo canal o tras toda la secuencia.**



Uso de las herramientas HAL

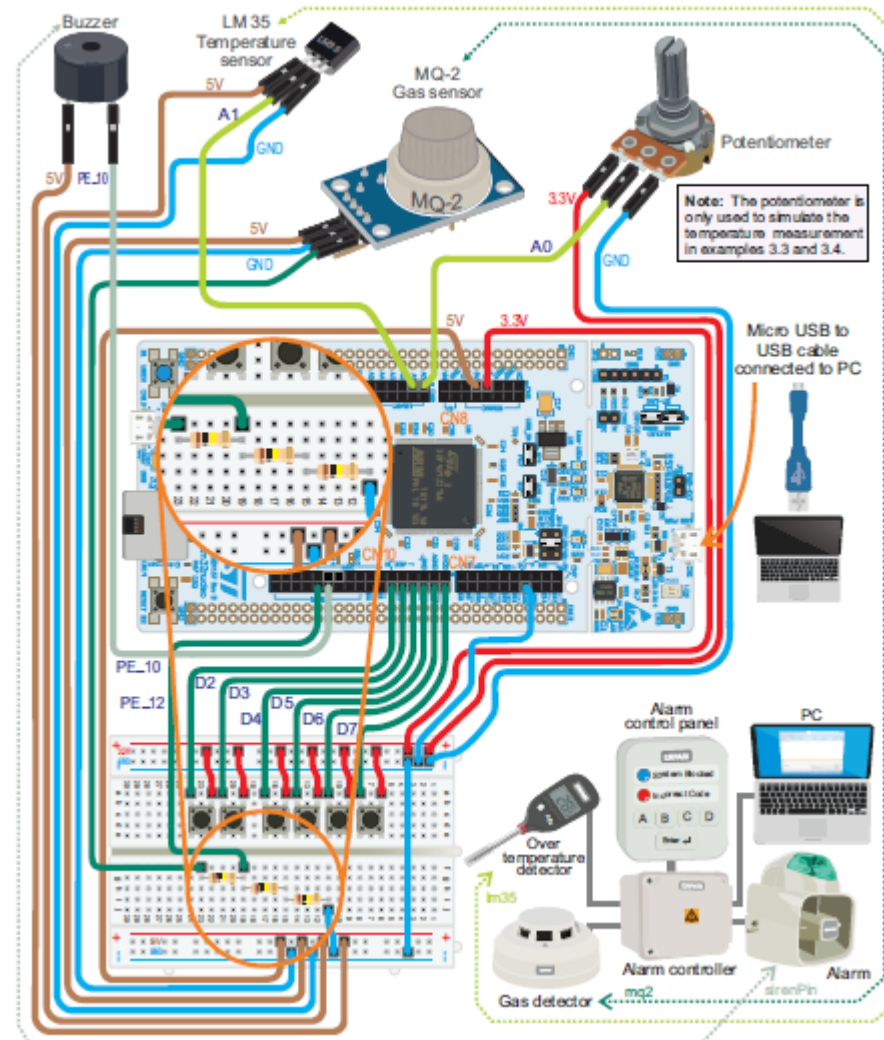


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.4 Uso de las herramientas HAL

```
#include <stdio.h>
#include "main.h"
#include "adc.h"
#include "usart.h"
#include "gpio.h"
uint32_t adc_raw; char msg[30];
void SystemClock_Config(void);
int main(void){ HAL_Init();
SystemClock_Config(); MX_GPIO_Init();
MX_ADC1_Init();
MX_USART2_UART_Init();
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
        adc_raw = HAL_ADC_GetValue(&hadc1);
        int len = sprintf(msg, "Valor ADC: %lu\r\n", adc_raw);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
    }
    HAL_Delay(1000);
}
}
```

3.4 Uso de las herramientas HAL

- **HAL_ADC_Start(ADC_HandleTypeDef* hadc)**
Inicia la conversión, si no, el ADC permanece en reposo.
- &hadc1: Es un puntero a una estructura de tipo Handle, contiene toda la identidad del ADC.

```
while (1){  
    HAL_ADC_Start(&hadc1);  
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){  
        adc_raw = HAL_ADC_GetValue(&hadc1);  
        int len = sprintf(msg, "Valor ADC: %lu\r\n", adc_raw);  
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);  
    }  
    HAL_Delay(1000);  
}
```

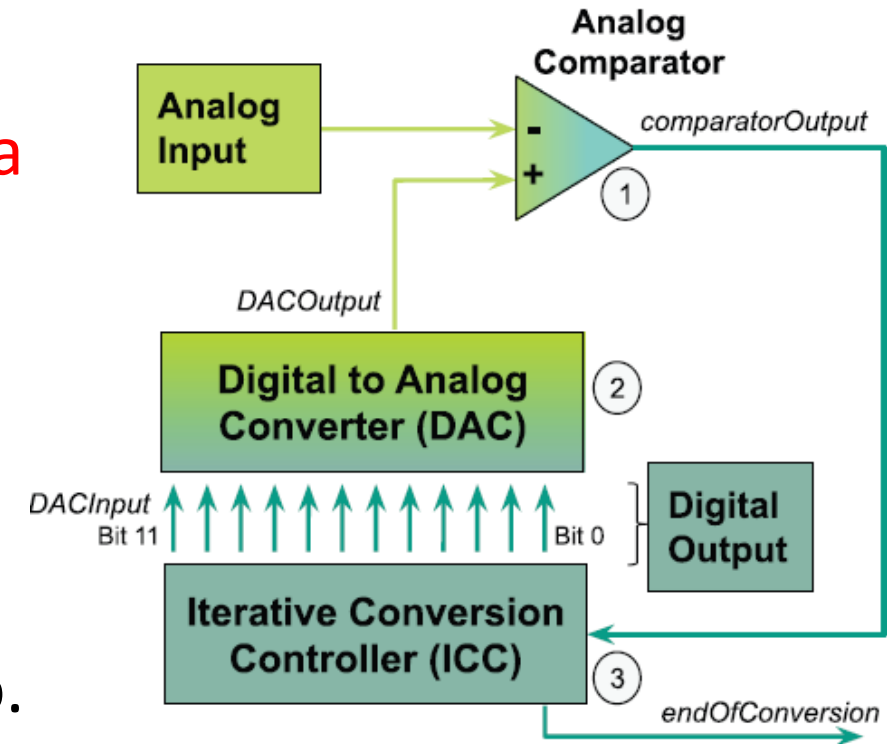
3.4 Uso de las herramientas HAL

- **HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc, uint32_t Timeout)**
- Función bloqueante, vigilar el hardware hasta que finaliza el ADC.
- Timeout: tiempo máximo (ms) que el procesador está dispuesto a esperar.

```
while (1){  
    HAL_ADC_Start(&hadc1);  
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){  
        adc_raw = HAL_ADC_GetValue(&hadc1);  
        int len = sprintf(msg, "Valor ADC: %lu\r\n", adc_raw);  
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);  
    }  
    HAL_Delay(1000);  
}
```


3.4 Uso de las herramientas HAL

- Si por algún error de hardware el ADC nunca termina, la función finaliza en Timeout.
- Internamente: entra en un bucle while que revisa constantemente la bandera EOC (End of Conversion) en el registro de estado.
- Cuando EOC se pone en 1, el bucle se rompe y el código continúa.



3.4 Uso de las herramientas HAL

- **HAL_ADC_GetValue(ADC_HandleTypeDef* hadc)**
- Al finalizar la conversión el dato se guarda en una variable.
- &hadc1: Le indica de qué registro de datos del ADC debe leer.
- Devuelve un valor de tipo uint32_t (0 a 4095).

```
while (1){  
    HAL_ADC_Start(&hadc1);  
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){  
        adc_raw = HAL_ADC_GetValue(&hadc1);  
        int len = sprintf(msg, "Valor ADC: %lu\r\n", adc_raw);  
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);  
    }  
    HAL_Delay(1000);  
}
```

Debug y Node-RED

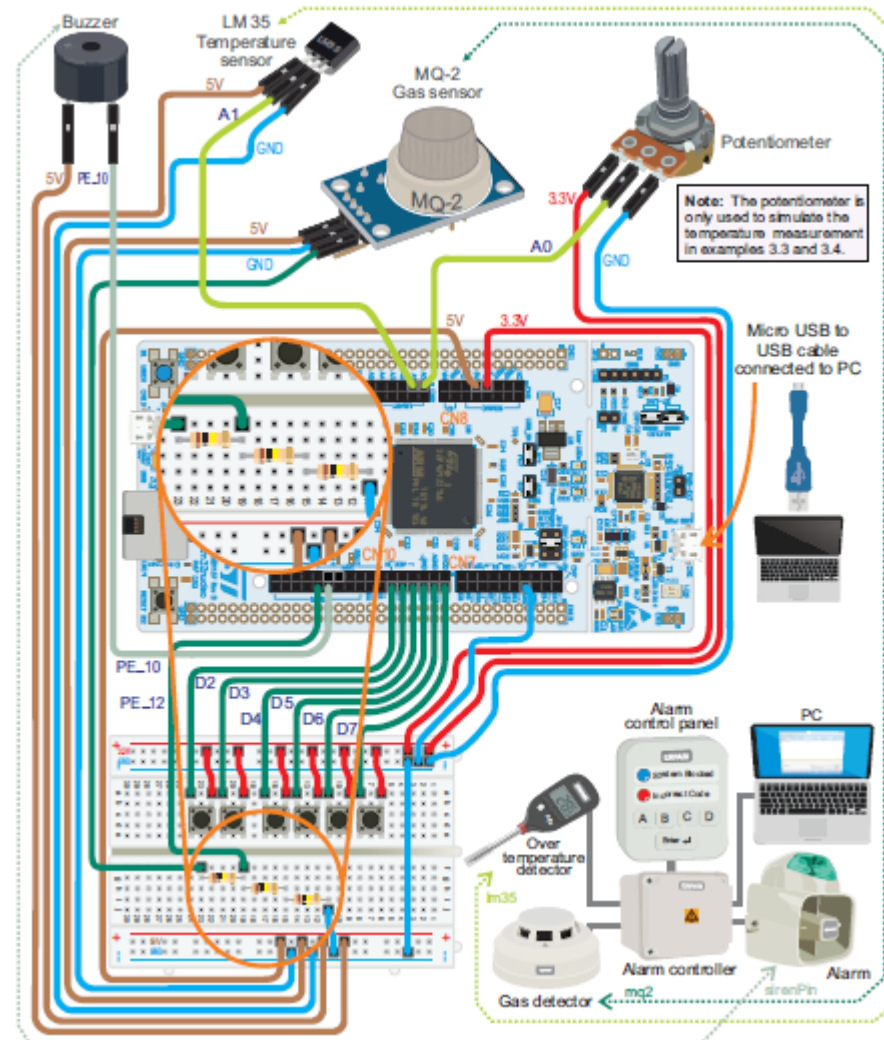
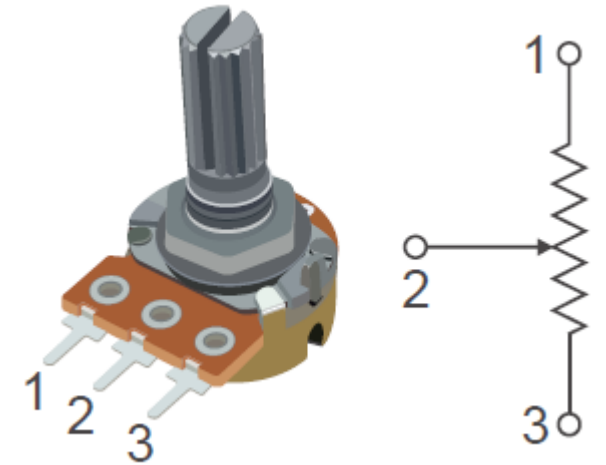


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

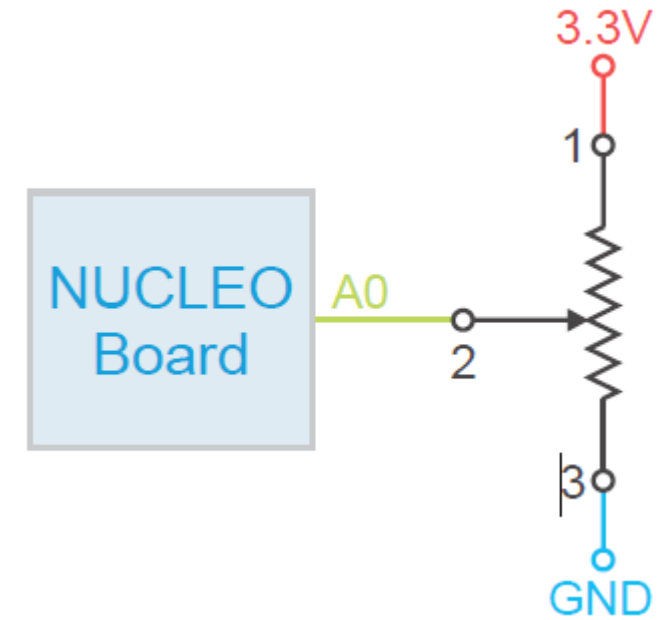
3.5 Debug y Node-RED

- Potenciómetro: permite variar la resistencia entre el terminal 2 y los terminales 1 y 3, según la posición angular de su perilla.
- Lo vamos a utilizar para explorar algunos conceptos del ADC.
- Figura: Un potenciómetro típico y su diagrama eléctrico.

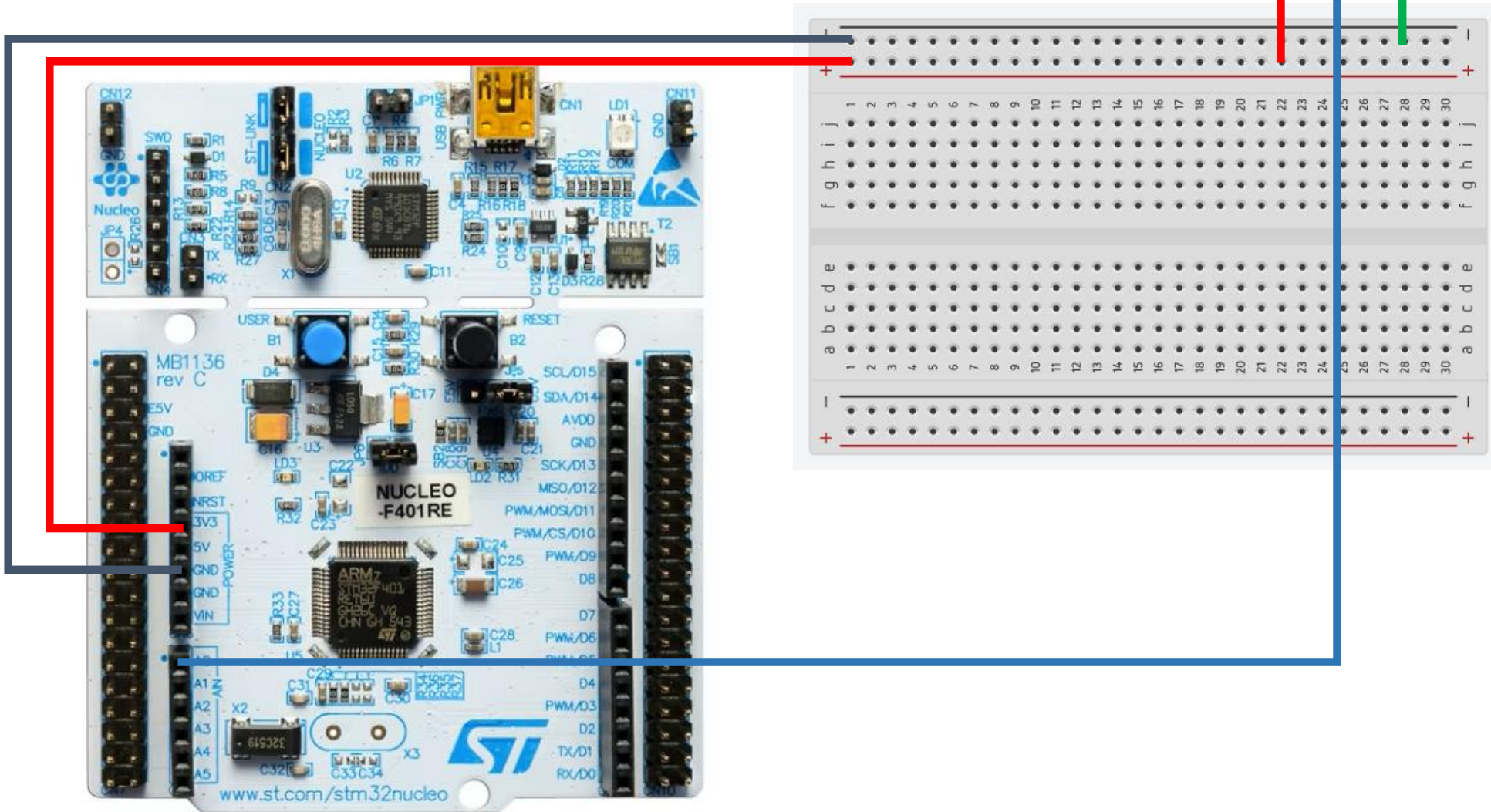


3.5 Debug y Node-RED

- Conexión potenciómetros – placa de desarrollo.
- Terminal 1 conectada a una tensión (3,3 V), terminal 3 conectada a GND y el terminal 2 conectado a una entrada analógica (A0).
- **NOTA:** Utilizar un potenciómetro de 10 kΩ. Si la resistencia es demasiado pequeña, la placa NUCLEO podría dañarse, y si es demasiado grande, la medición podría ser inestable.

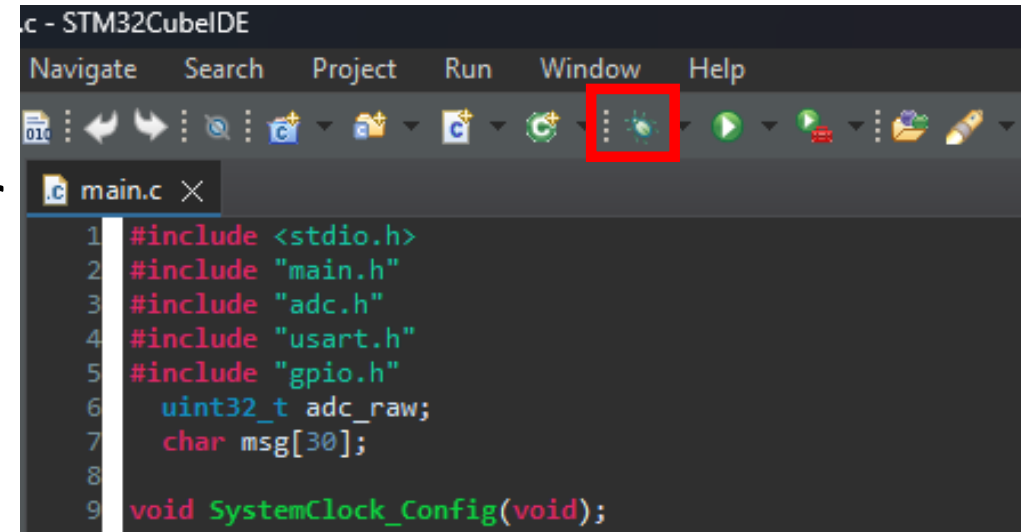


3.5 Debug y Node-RED



3.5 Debug y Node-RED

- **Debug:** Proceso que permite supervisar y controlar la ejecución del código para dar respuesta a tres preguntas críticas:
- ¿Por qué mi programa se queda trabado en una línea?
- **¿Qué valor tiene mi variable realmente en este instante?**
- ¿Está entrando mi código al if de la alarma o se lo está saltando?



3.5 Debug y Node-RED

- Se usa el ST-LINK que tiene la placa Nucleo:
- El Debug tiene diferentes funciones, en esta etapa del curso se utiliza **Live Expressions**.
- Permite ver el o los valores de tus variables mientras el programa corre, sin necesidad de usar un printf o una pantalla LCD.

ST-LINK part



Target MCU part



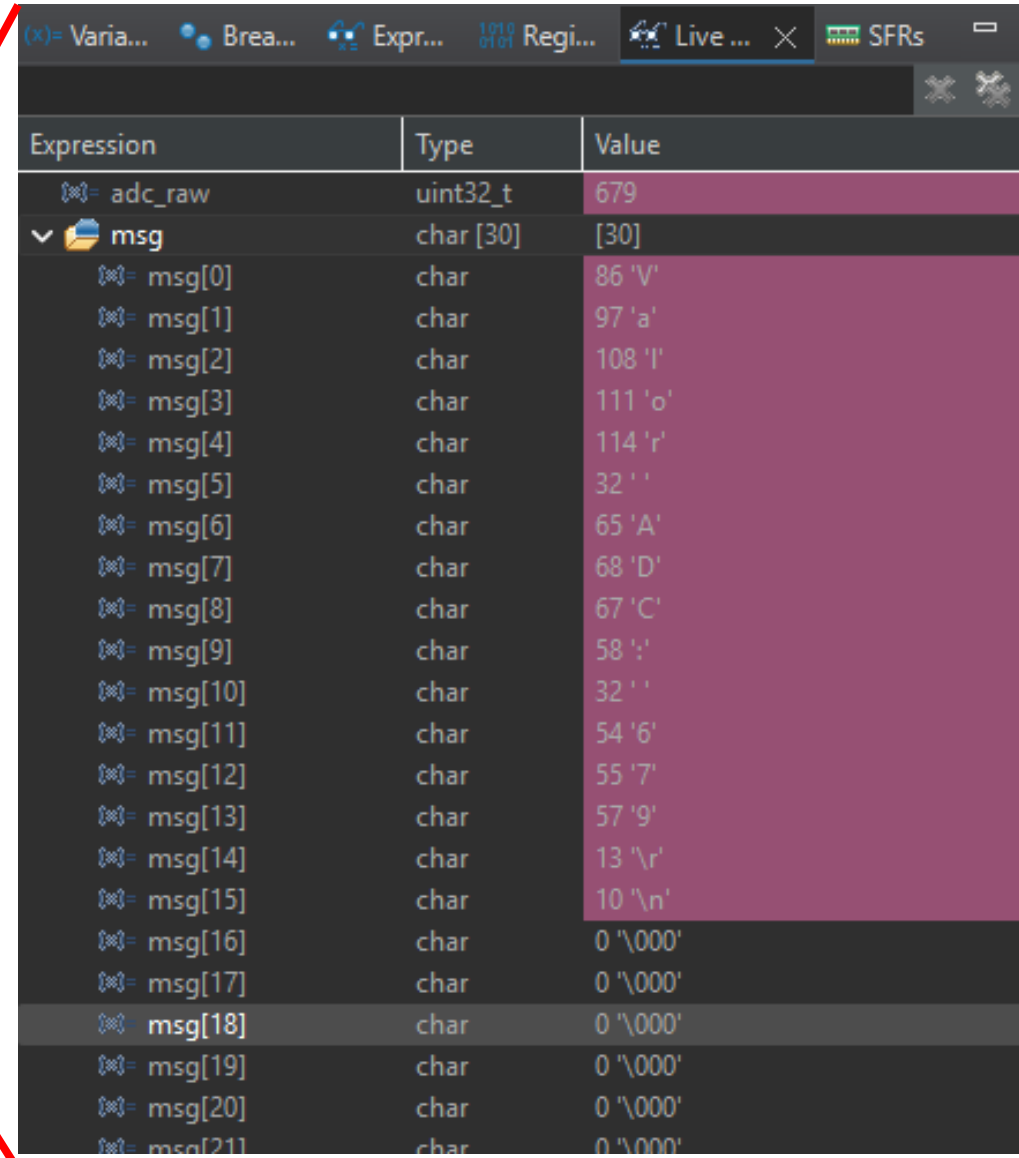
3.5 Debug y Node-RED

- En el ejemplo, el **Live Expressions** se observa de la siguiente forma.

```
#include <stdio.h>
#include "main.h"
#include "adc.h"
#include "usart.h"
#include "gpio.h"
uint32_t adc_raw;
char msg[30];

void SystemClock_Config(void);

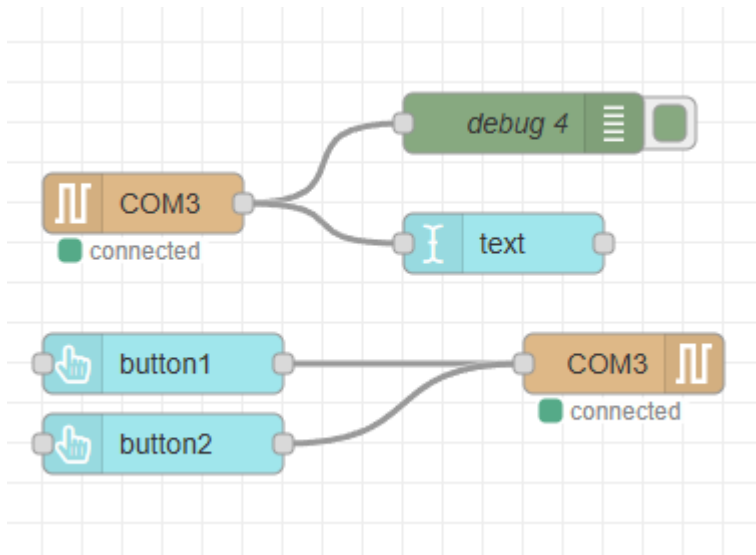
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_USART2_UART_Init();
    while (1){
        HAL_ADC_Start(&hadc1);
        if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
            adc_raw = HAL_ADC_GetValue(&hadc1);
            int len = sprintf(msg, "Valor ADC: %u\r\n", adc_raw);
            HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
        }
        HAL_Delay(1000);
    }
}
```



Expression	Type	Value
{*}= adc_raw	uint32_t	679
▼ {*}= msg	char [30]	[30]
{*}= msg[0]	char	86 'V'
{*}= msg[1]	char	97 'a'
{*}= msg[2]	char	108 'l'
{*}= msg[3]	char	111 'o'
{*}= msg[4]	char	114 'r'
{*}= msg[5]	char	32 ' '
{*}= msg[6]	char	65 'A'
{*}= msg[7]	char	68 'D'
{*}= msg[8]	char	67 'C'
{*}= msg[9]	char	58 ':'
{*}= msg[10]	char	32 ' '
{*}= msg[11]	char	54 '6'
{*}= msg[12]	char	55 '7'
{*}= msg[13]	char	57 '9'
{*}= msg[14]	char	13 '\r'
{*}= msg[15]	char	10 '\n'
{*}= msg[16]	char	0 '\000'
{*}= msg[17]	char	0 '\000'
{*}= msg[18]	char	0 '\000'
{*}= msg[19]	char	0 '\000'
{*}= msg[20]	char	0 '\000'
{*}= msg[21]	char	0 '\000'

3.5 Debug y Node-RED

- Paso 1: Utilizaremos primero flujo realizado con Node-RED en el Example 1.3.



≡ Page 2

Group 3

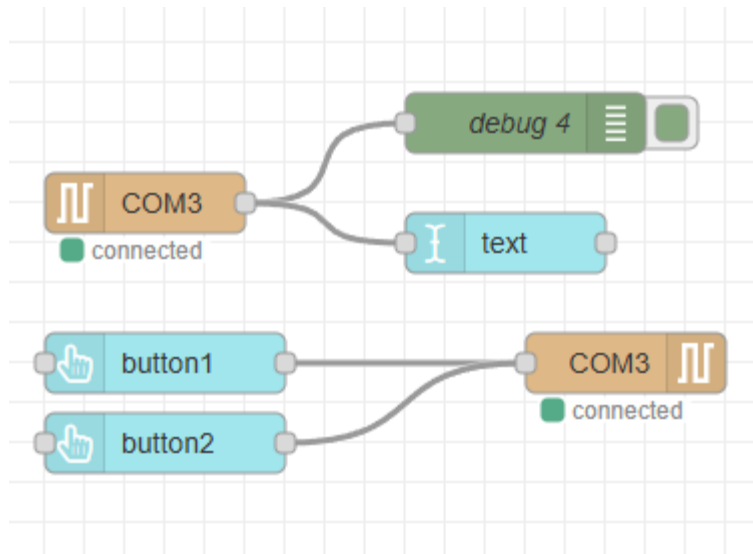
button2

text
The alarm is activated

button1

3.5 Debug y Node-RED

- Información del Debug de Node-RED.



debug

all nodes all

15/5/2026, 14:37:33 node: debug 4
msg.payload : string[16]
▶ "Valor ADC: 3394"

15/5/2026, 14:37:34 node: debug 4
msg.payload : string[16]
▶ "Valor ADC: 3384"

15/5/2026, 14:37:35 node: debug 4
msg.payload : string[16]
▶ "Valor ADC: 3414"

15/5/2026, 14:37:36 node: debug 4
msg.payload : string[16]
▶ "Valor ADC: 3394"

15/5/2026, 14:37:37 node: debug 4
msg.payload : string[16]
▶ "Valor ADC: 3374"

Problemas

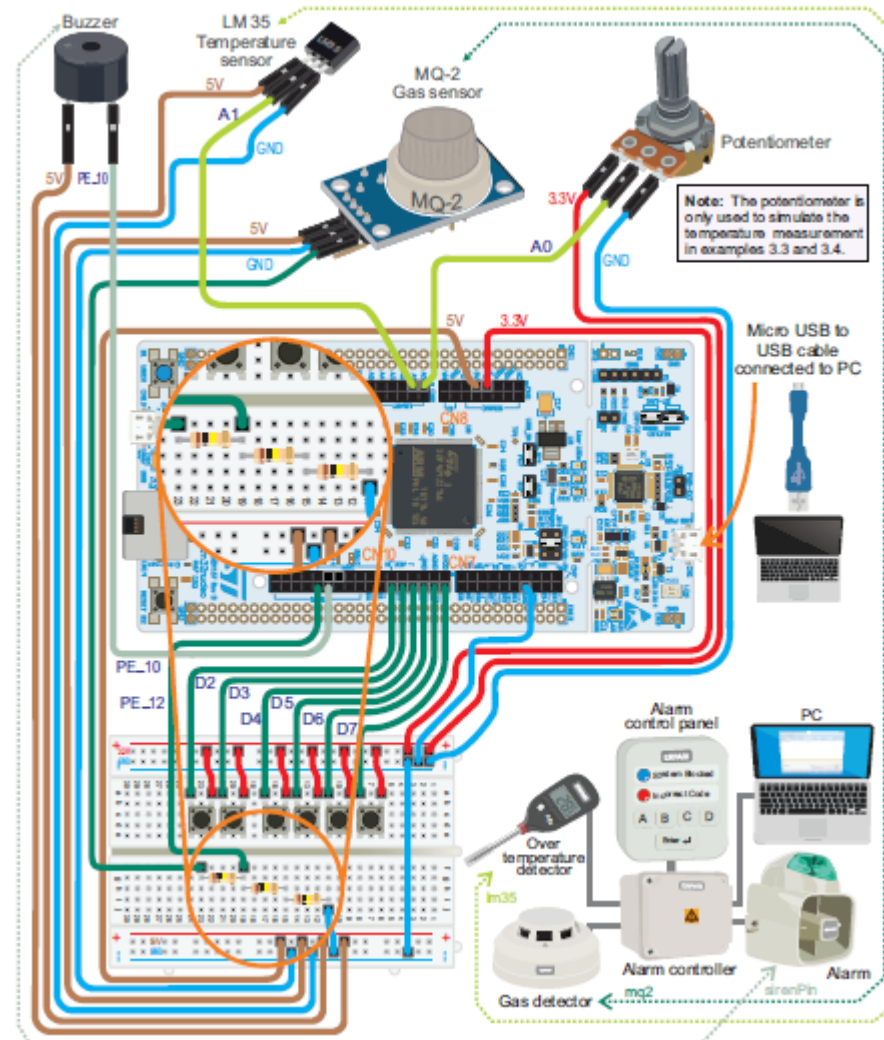
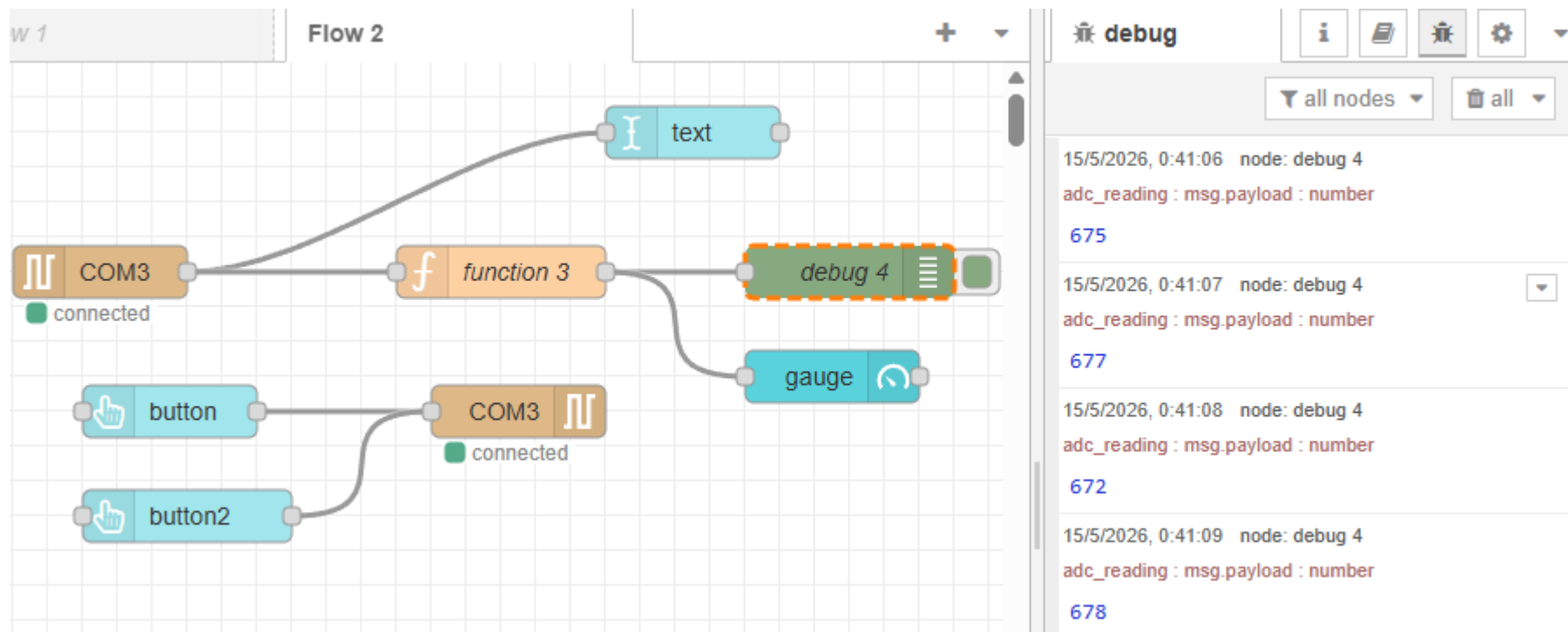


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

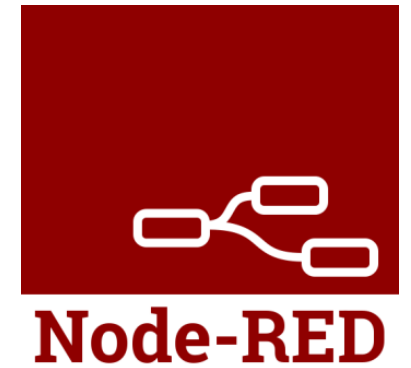
<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.6 Problemas

- Paso 2: Aplicaremos el nodo function y el nodo gauge.



3.6 Problemas



- En el nodo function aplicaremos el siguiente código:

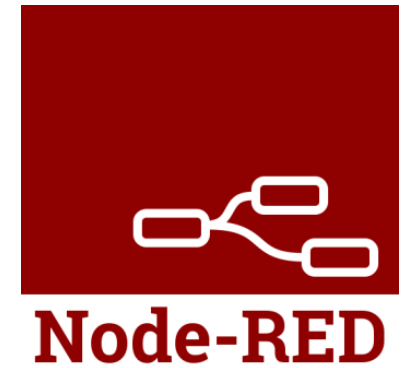
```
let rawData = msg.payload.toString();
let match = rawData.match(/\d+/);
if (match) {
    let adcValue = parseInt(match[0], 10);
    msg.payload = adcValue;
    msg.topic = "adc_reading"; return msg;
} else {
    return null;
}
```

3.6 Problemas

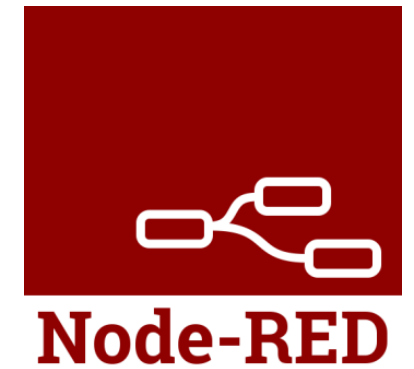


- El código en el nodo Function se escribe en JavaScript.
- JavaScript es el lenguaje de la web, flexible, dinámico y es el que utiliza Node.js (el entorno sobre el que corre Node-RED).
- JavaScript es excelente en el manejo de eventos y utiliza los JSON de forma nativa.
- No necesitas compilar.

3.6 Problemas

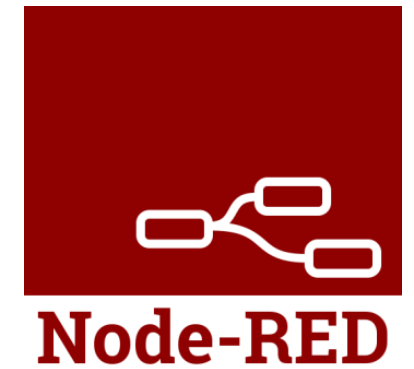


- Cuando se envían datos desde una STM32 por UART, a menudo llegan con texto adicional(\n o \r).
- El nodo function actúa como un filtro y el Dashboard solo recibe el número puro.
- `let rawData = msg.payload.toString();`
- Recibe los datos de la STM32 usualmente como un Buffer (cadena de bytes), esos bytes se convierten en una "cadena de texto" (String).



3.6 Problemas

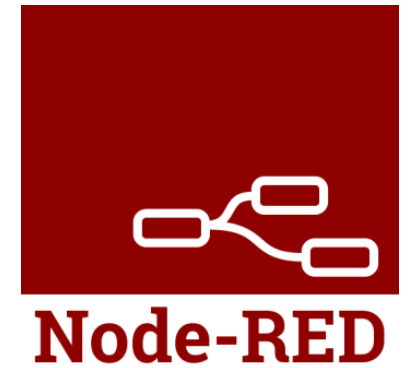
- `let match = rawData.match(/\d+/);`
- `.match()` busca dentro del texto siguiendo una Expresión Regular, `\d`: Significa "busca cualquier dígito del 0 al 9".
- `+`: Significa "y sigue buscando si hay más números juntos".
- Ejemplo: Si tu STM32 envió "ADC: 2048\r\n", esta línea ignora el "ADC: ", el espacio y los caracteres extra, y captura solo el "2048".



3.6 Problemas

- Conversión de Texto a Número:
- `if (match)`: Si el mensaje llegó vacío, evita que el código falle.
- `match[0]`: La función `match` devuelve una lista (array). El primer elemento (`[0]`) es el número que encontró.
- `parseInt(..., 10)`: Convierte el texto "2048" en el número real 2048. El 10 indica que estamos usando el sistema decimal (base 10).

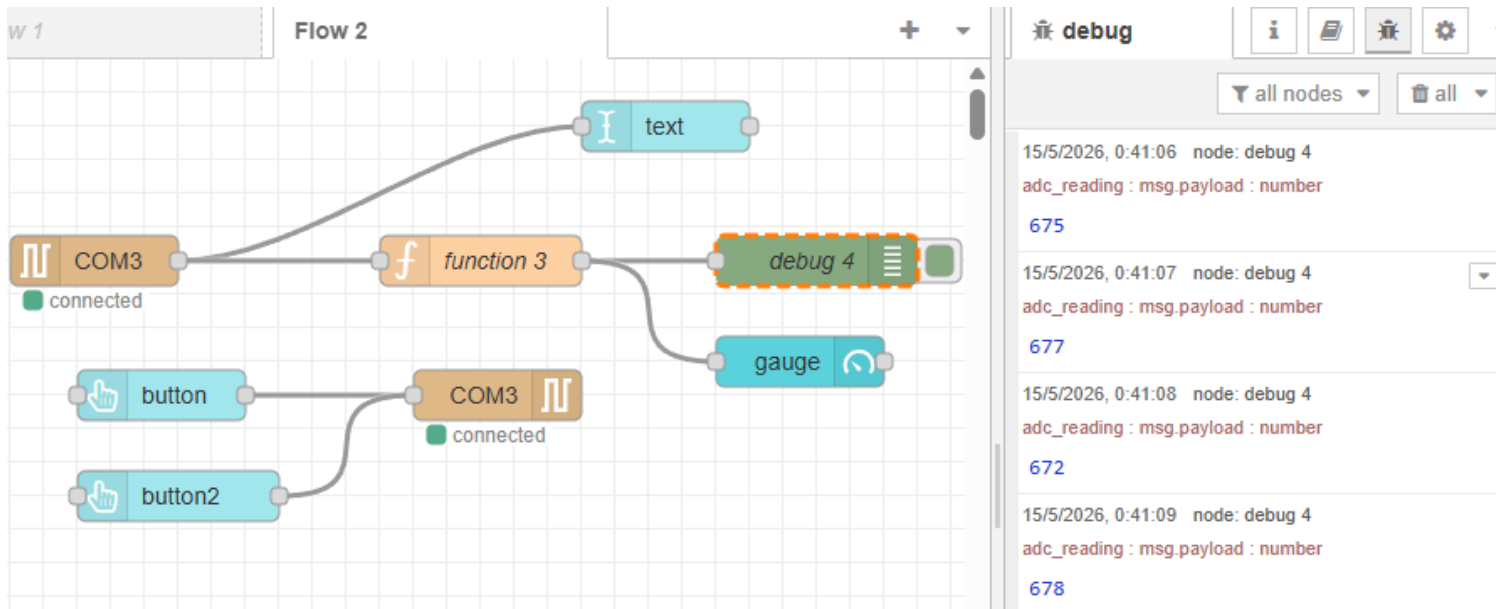
3.6 Problemas



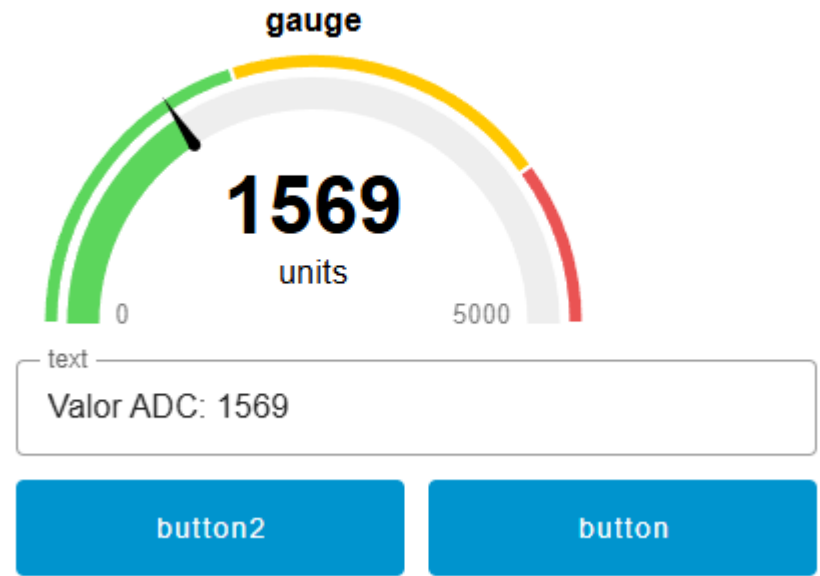
- Empaquetado para el Dashboard:
- `msg.payload = adcValue;`
- `msg.topic = "adc_reading";`
- `return msg;`
- Sobrescribimos el mensaje original, solo el número limpio.

3.6 Problemas

- Salida final.

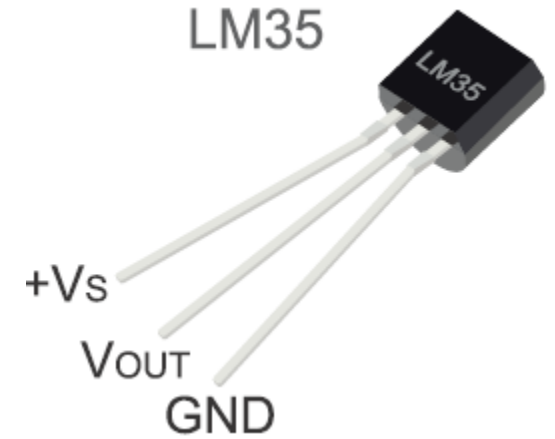


Group 3



3.6 Problemas

- Trabajaremos con el sensor de temperatura LM35, este debe conectarse a 5 V.
- Para conectar módulos y elementos, desconecte la fuente de alimentación USB de la placa NUCLEO y antes de volver a conectarla, compruebe que las conexiones estén realizadas correctamente y de forma segura.



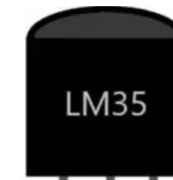
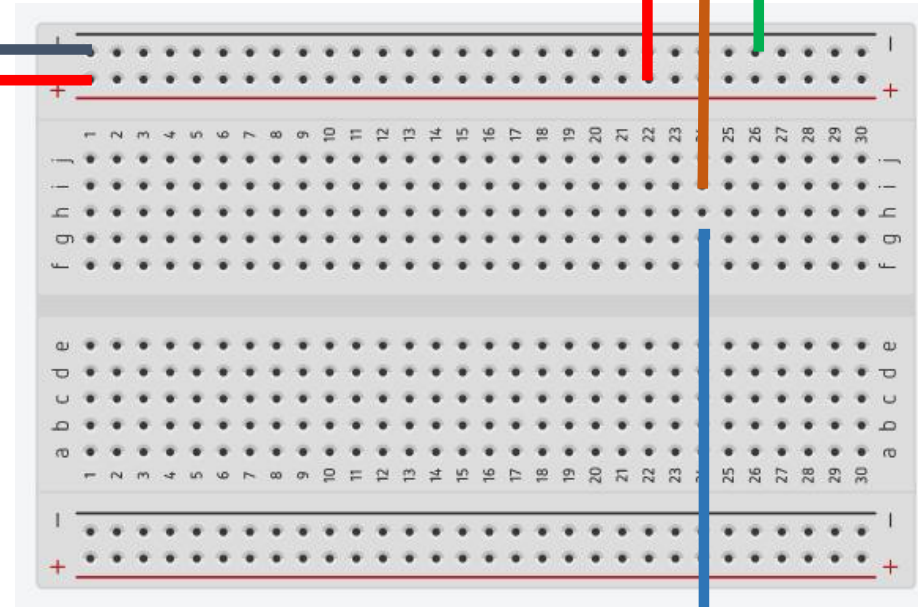
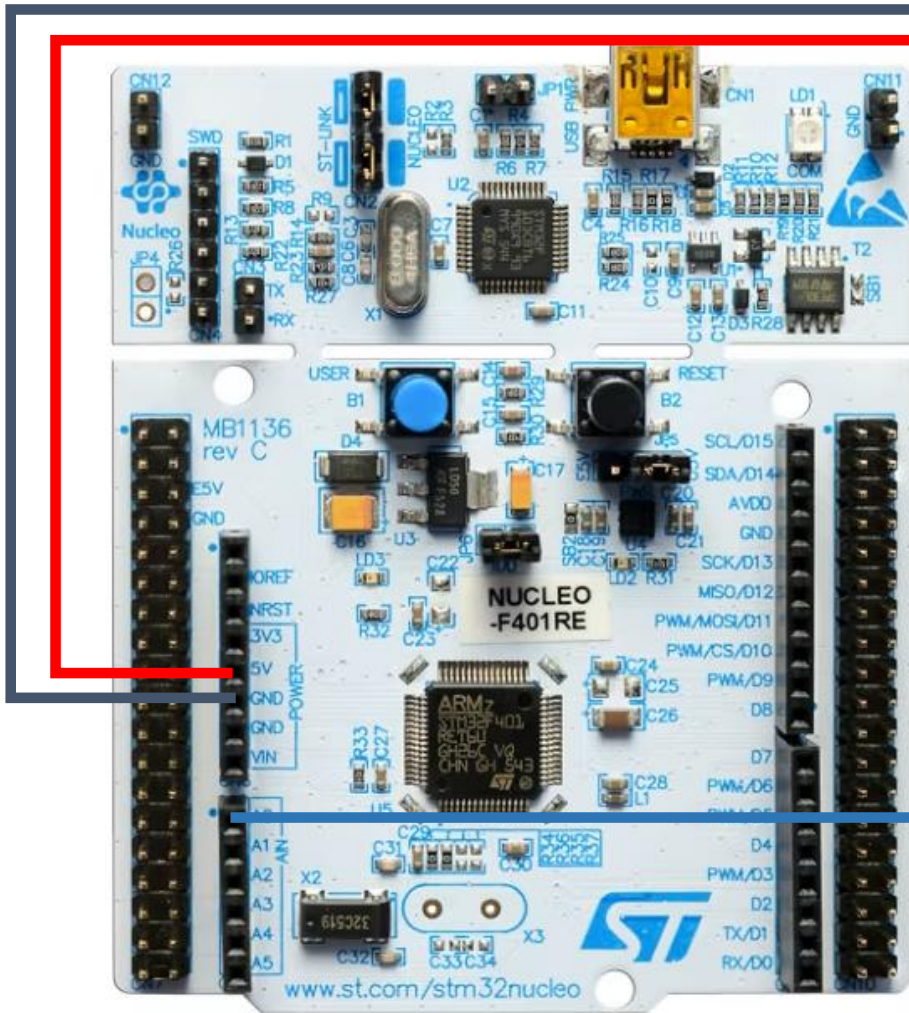
3.6 Problemas

- Algunas características del sensor.

Features

- Calibrated directly in ° Celsius (Centigrade)
- Linear + 10.0 mV/°C scale factor
- 0.5°C accuracy guaranteeable (at +25°C)
- Rated for full -55° to +150°C range
- Suitable for remote applications
- Low cost due to wafer-level trimming
- Operates from 4 to 30 volts
- Less than 60 µA current drain
- Low self-heating, 0.08°C in still air
- Nonlinearity only $\pm 1/4^\circ\text{C}$ typical
- Low impedance output, 0.1 Ω for 1 mA load

3.6 Problemas

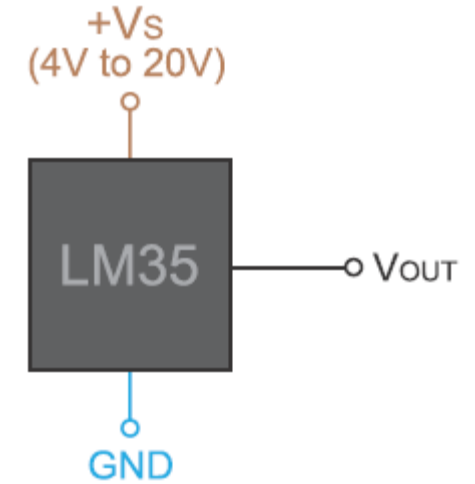


3.6 Problemas

- Modificaremos el ejemplo del inciso 3.4.

```
#include <stdio.h>
#include "main.h"
#include "adc.h"
#include "usart.h"
#include "gpio.h"
uint32_t adc_raw;
char msg[30];
float Temp;
void SystemClock_Config(void);
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_ADC1_Init();
    MX_USART2_UART_Init();
```

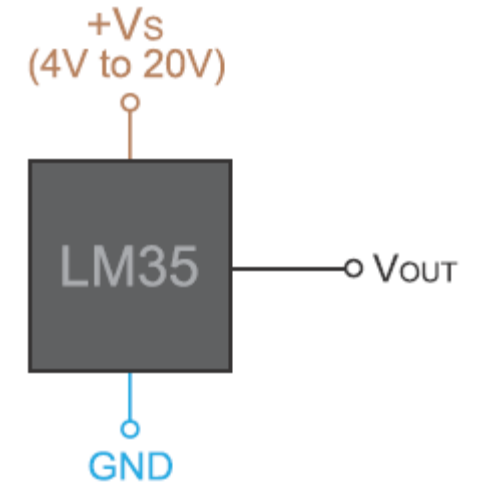
```
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
        adc_raw = HAL_ADC_GetValue(&hadc1);
        Temp = adc_raw * 3.3 / (4095.0*0.01);
        int tnode = (int)(Temp * 10.0 + 0.5);
        int len = sprintf(msg, "TEMP: %d\r\n", tnode);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
    }
    HAL_Delay(1000);
}
```



3.6 Problemas

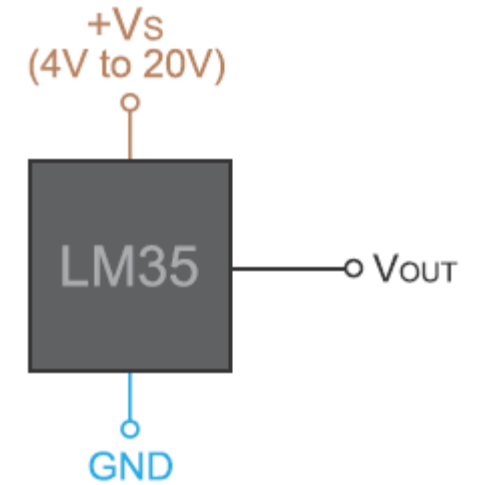
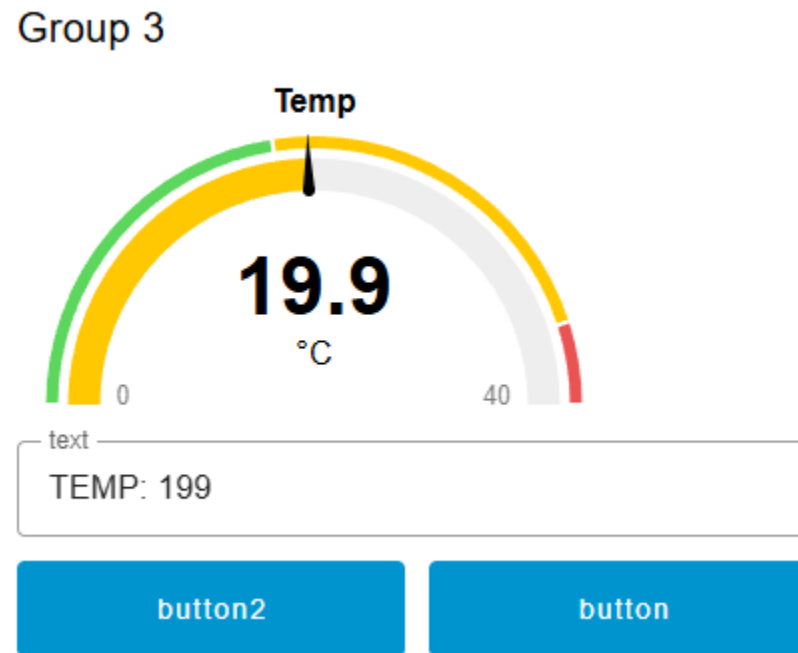
- Es importante notar que se envía un entero.
- No se utiliza la FPU (Floating Point Unit).

```
while (1){
    HAL_ADC_Start(&hadc1);
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK){
        adc_raw = HAL_ADC_GetValue(&hadc1);
        Temp = adc_raw * 3.3 / (4095.0*0.01);
        int tnode = (int)(Temp * 10.0 + 0.5);
        int len = sprintf(msg, "TEMP: %d\r\n", tnode);
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, len, 100);
    }
    HAL_Delay(1000);
}
```



3.6 Problemas

- Modificamos Node-RED y observamos lo siguiente.



3.6 Problemas

- En el libro de referencia, en el Code 3.4 se observa lo siguiente.
- Modificar el código del último ejemplo de clase para incorporar el ADC al proyecto.

```
1 void alarmActivationUpdate()
2 {
3     potentiometerReading = potentiometer.read();
4
5     if ( potentiometerReading > POTENTIOMETER_OVER_TEMP_LEVEL ) {
6         overTempDetector = ON;
7     } else {
8         overTempDetector = OFF;
9     }
10
11     if( gasDetector) {
12         gasDetectorState = ON;
13         alarmState = ON;
14     }
15     if( overTempDetector ) {
16         overTempDetectorState = ON;
17         alarmState = ON;
18     }
19     if( alarmState ) {
20         accumulatedTimeAlarm = accumulatedTimeAlarm + TIME_INCREMENT_MS;
21
22         if( gasDetectorState && overTempDetectorState ) {
23             if( accumulatedTimeAlarm >= BLINKING_TIME_GAS_AND_OVER_TEMP_ALARM ) {
24                 accumulatedTimeAlarm = 0;
25                 alarmLed = !alarmLed;
26             }
27         } else if( gasDetectorState ) {
28             if( accumulatedTimeAlarm >= BLINKING_TIME_GAS_ALARM ) {
29                 accumulatedTimeAlarm = 0;
30                 alarmLed = !alarmLed;
31             }
32         } else if ( overTempDetectorState ) {
33             if( accumulatedTimeAlarm >= BLINKING_TIME_OVER_TEMP_ALARM ) {
34                 accumulatedTimeAlarm = 0;
35                 alarmLed = !alarmLed;
36             }
37         }
38     } else{
39         alarmLed = OFF;
40         gasDetectorState = OFF;
41         overTempDetectorState = OFF;
42     }
43 }
```

Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media